FlexMan ANN Hardware Accelerator: Scheduler and Programming Model

Author: Simon Davidson, 1/4/2026

Version: 2.0

Contents

Overview

The name ***FlexMan*** refers to a set of one or more hardware accelerators targeting neural network applications, both artificial (ANN) and spiking (SNN). Each such accelerator is capable of executing the operations to perform the compute for a single layer of neurons.

The system architecture also includes a single Scheduler whose task it is to ensure that an entire neural network of many layers is executed using the finite hardware by sending the data for each layer (weights, input activations and bias currents) in turn to one of the accelerators for processing and capturing the resulting output to be used as input to other layers.

The definition of the layers, their input and output data streams and the order in which they should be processed is conveyed using a program of 32-bit and 64-bit instructions. This instruction stream is read from an external memory and processed in the scheduler. As all data (for both the

input to and the output from each layer) is stored in number buffers, instructions only need to refer to data by its buffer number. The scheduler tracks which buffers are empty (and free to be filled by the output of new processing), which are complete (an operation has completed and the output has been captured by the output buffer) or busy (the buffer is currently being filled by an operation ongoing in one of the hardware accelerators.

The scheduler keeps a table of instructions (called tasks) that it has read from memory but which have not yet been sent to an accelerator. For each task, it retains a list of which buffers it requires as input and the buffer it needs to send its output to. For each such entry there is also a field indicating to which accelerator the instruction must be sent, When a waiting task has all input buffers full and output buffer free, the scheduler will send that task to the target accelerator.

The state used to managed these functions is described later.

Input Buffers

All data used by the accelerators: input weights and activations, bias currents threshold information, is stored in SRAM external to the scheduler and accelerators. To avoid passing around address pointers to this information, each buffer has an ID called the buffer number. So a task may take its input from buffers 1 and 6 and send its output to buffer 4, for example. The start addresses for these buffers are stored in a table external to the scheduler and accelerators, with the buffer base address being passed to the accelerator when the task is loaded. This translation is handled by logic external to the scheduler.

HW Config Store

The individual accelerators are flexible, so that, for example, the layer processor can work with activations that are any power of two in bit width, from a single bit up to 32-bit. Similarly, the weights can be of any integer size from 1-bit to 32-bit. Since a single layer processing compute module may work with any activation-weight pairing, it is necessary to pass the operand sizes (and other information) to the accelerator unit before processing of each layer begins. This information is fixed for the processing of one layer, but may change for the next.

Rather than embed the configuration information in the instruction stream, a separate configuration store is present, which includes the settings for one of the virtual layers of the network. As part of the TASK instruction which performs the execution of a single layer of the network, the user can specify the hardware configuration to use by referring to its index in stable. Thus the config must be pre-loaded. The config_index field in each TASK instruction acts as an index into a dedicated table of configurations. A configuration is one or more 32-bit words, which are loaded into the hardware accelerator on the cycle in which that the task is loaded up to the compute unit. They must be set up by an external agent via the AXI bus that tethers the accelerator to the main CPU. The accelerator is memory mapped, with the layout of the mapping, including the config registers, given at the end of this document.

Scheduler State and Operation

This section begins by describing the state that must be held by the scheduler to perform its functions. Next it describes the normal operation of the scheduler, which explains how this state is used to manage correct execution of the target neural network. This sections also illustrates the different commands of the scheduler's programming language, which are presented formally in the next section.

State Held by the Scheduler

There are two types of resource that must be managed and deployed appropriately:

- The hardware accelerators themselves, which perform the computation
- The storage buffers which hold the input data and receive the output data of each task.

Each new task requires one or more buffers and (optionally) a weight matrix to supply the input and one or more empty output buffers to receive the result of the computation. The task must be allocated to one of the accelerator blocks, which is then busy (and unavailable for other task) until the task is complete. The scheduler must track both the usage of the individual accelerators as well as the status of the buffers.

Each compute unit has a 1-bit flag associated with it in the scheduler, which indicates if it is free to receive a new task (value 0) or currently occupied with a task (value 1). A task can only be assigned to a compute unit if its busy flag is clear.

Each buffer has four pieces of state associated with it. First, a busy signal that is set when a buffer will be used as the output of task but is not yet complete. This is used to stall the addition of any new tasks to the table that will target the same buffer, which represents an out-of-order hazard.

Second, a full signal that is set when a task has completed and placed data in the buffer. This indicates that the buffer can be consumed by any tasks that are waiting for it.

The third state associated with each buffer is a colour bit. This is used to ensure that the output of a task going to multiple places is correctly distributed and avoids any issues with later task that happen to be delayed in arriving in the scheduler table finding that their needed data has been consumed by another task (incorrectly).

The fourth and final state associated with each buffer is a counter that is set when a task is added to the table, indicating how many other tasks will want to consume the buffer once it is full. The initial value of the counter comes from the instruction and is used to know when the buffer contents no longer need to be retained. When the counter reaches zero the full bit is cleared and any instructions waiting to target that buffer can be placed in the table and initiated. The colour bit for this buffer is flipped so that it alternates in colour.

Module Definition for the Scheduler

The module definition for the scheduler consists of the following signals:

Name	In/Out	Size	Notes
clk	In	1	
reset	In	1	
Interface to AXI bus:			
sys_req_i	In	1	
sys_ack_o	Out	1	
sys_data_i	In	32	
sys_data_o	Out	32	
Control Interface:			
start_program_i	In	1	
program_addr_i	In	PROG_ADDR_BITS	

prog_mem_addr_o	Out	ADDR_SIZE	Address of instruction in program memory
prog_mem_data_i	In	PROG_DATA_BITS	Data for requested instruction
prog_mem_req_o	Out	1	Assert to request data from program memory
prog_mem_wait_i	In	1	Asserted if the requested data will not be valid on the next cycle
Interface to Accelerators:			
acc_busy_i	In	NUM_ACC	one-hot signal from every accelerator, high if that unit is busy, low if it is free
acc_finished_i	In	NUM_ACC	Single cycle high pulse on the cycle in which the accelerator has completed its current task and is free to receive a new one
start_new_block_o	Out	1	Asserted when a new task is sent to an accelerator
target_acc_o	Out	TGT_ACC_SZ	ID of the target accelerator
buffer_info_o	Out	SCH_ENTRY_SZ	

xxx

Scheduler Operation

The scheduler runs the user's network as a program: a list of tasks to process individual layers of the network. To begin, a start signal is asserted for a cycle, together with the start address of the network program in external memory. These signals are input to the scheduler. This initiates fetching at the given address, which must be 32-bit aligned. Commands are either 32-bit or 64-bit (currently).

The scheduler fetches commands and places them in order into a table (currently with 16 entries). Any entry in the top four can be chosen to be sent to one of the compute blocks provided all of the following conditions are met:

1) The target accelerator is free
2) All required source buffers are marked as full in the scheduler buffer tracking table and all of their *colours* match the colour of the instruction (if not, the buffer content is intended for a different task and cannot be taken)
3) The required destination/target buffer is marked as free in the scheduler buffer tracking table

When an entry is sent to a hardware block it is removed from the table and the usage counter for each of the source buffers is decremented. The busy bit for the target compute block is set to indicate that this block is not available for a new task at the moment.

Operation continues until the scheduler receives a STOP command, at which point is ceased to add entries to the table, awaits the completion of all outstanding tasks and then stops, sending a finished signal back to the host CPU on the system bus as a last action. It can then be awakened by another program, which resets all buffer state (marks all buffers and compute blocks as free).

Programming Model: Network Specification

This section outlines the Instruction Set Architecture (ISA) for the FlexMan scheduler. The encoding for the command is subject to change between now and the end of the project.

Command Format

Commands consist of one or more 32-bit data words aligned to 32-bit boundaries.

Bits[3:0] : Opcode, defines what the operation is.

Bits[31:4]: Operation dependent

Bits[63:32]: Optional. The presence of this second word depends on the opcode. If bit 2 of the opcode is set, this indicates a two-word instruction.

List of Commands

The following commands are currently supported by the scheduler. Only FILL is currently a 64-bit instructions. All others are 32-bit.

TASK Command (opcode 3'b000)

Describes a task to be carried out by one of the compute blocks. The fields are:

- Source buff 1 and 2: the sources of any input data (using the ID of the buffers in which they are stored),
- Target buffer: the destination buffer into which results will be sent
- Colour: The 'colour' of this task, so that it only uses buffer of the same colour to ensure correct data flow
- #Targets: The number of other tasks that will consume the output of this task. Used to track buffer use and obsolescence
- Acc. ID: The identity of the hardware compute block that must be assigned the task
- Config ID: An identifier to select from a list of configuration values to be load to the config registers of the hardware at the initiation of the task. This permits a single flexible compute block to switch between different modes and/or operand bit ranges. Config settings must be loaded up in advance.

29:25	24:20	19:17	16:12	11:9	8	7:3	2:0

Source buff 2	Source buff 1	Acc. ID	Config ID	#Targets	Colour	Target buffer	Opcode

JUMP Command (opcode 3'b001)

Continue fetching schedule tasks, starting from a new location in memory. Address is taken to be a multiple of 32-bit, so will correspond to a new line in a 32-bit memory. Jump is unconditional and the address is absolute.

31:3	2:0
Jump Address	Opcode

Stop Command (opcode 3'b010)

Do not read any more instructions or start any commands that are after the stop command in the instruction stream. Wait until all outstanding tasks have completed. Terminate operations and return '*finished'* to the parent CPU via the system bus.

	2:0
	Opcode

Check Command (opcode 3'b011)

Used to support dynamic neural networks (DyNN), where the flow of tasks can be modified as the result of the processing in one of the previous tasks.

Two modes supported:

- *finish on success*, where a successful check causes the program to terminate and the results can then be read from the output buffer of the last task (bit[3] = 1)
- *skip on success*, where a successful check causes the program to jump to a new point in the program, typically used to bypass several layers in the network (bit[3] = 0)

In either mode, a *check fail* indicates that the program continues with the next command in sequence, just as with a traditional failed conditional branch or jump.

In the *skip on success* mode, success loads the fetch counter with a 32-bit aligned value specified in the skip address field. In *the finish on success* mode, network operation terminates and all task of the same colour as the check command (specified in the colour bit in the instruction) are cancelled and no new task are issued until the accelerator receives a continue signal from an external source. This permits an external agent to retrieve any needed buffer contents before they are overwritten by continued processing.

The success/fail indicator comes from the output of another task, as nominated in bits[10:4] of the check instruction. Some hardware compute units have a prime function to produce a success/fail indicator as their output.

31:12	11	10:4	3	2:0
Skip Address	Colour	Check Unit ID	Finish on successor OR skip on success	Opcode

Fill Command (opcode 3'b101)

This instruction is 64-bit, requiring two 32-bit words from the program memory.

Pre-loads a buffer from a block of external memory. Used to set up the first buffer in recurrent networks such as GRU.

The command indicates the ID of the *target buffer* into which the data will be loaded and the *colour* the buffer will assume (used to correctly match the buffer contents to the task command that will consume it). It also indicates (through the *#Targets* field) the number of tasks that will consume the given data, used to decide when the buffer content is obsolete and so the buffer itself can be freed for future tasks to use.

The first command word also supplies the number of 32-bit words to be loaded. A second 32-bit words provides the 32-bit aligned start address in external memory where the buffer contents can be found.

Command word 1:

31:12	11:9	8	7:3	2:0
Block size (32-bit words)	#Targets	Colour	Target buffer	Opcode

Command word 2:

31:2	1:0
Block start address (32-bit aligned)	0b00

Loop Command (opcode 3'b110)

Forms a pair with the LoopEnd command. Indicates that the following block of commands will be repeated, for a number of times specified in the Max loop value field. As loops can be nested, a 3-bit loop ID field (bits [5:3]) is used to identify the loop. Thus up to 8 loops could be embedded.

The Loop instruction causes the loop count associated with the given Loop ID to be set to the Max loop value, along with the PC of the following instruction (called the loop restart address). When the LoopEnd instruction is received, the loop counter is decremented and instruction fetching continues with loop restart address associated with that Loop ID.

31:6	5:3	2:0
Max loop value	Loop ID	Opcode

LoopEnd Command (opcode 0b111)

This command marks the end of a loop started with the Loop command (opcode 3'b110). It will trigger a decrement of the loop counter for that ID. When the loop counter reaches zero, instruction fetch continues with the PC after the LoopEnd instruction. If the counter is not zero, fetch jumps to the Loop restart address, associated with the Look ID.

31:6	5:3	2:0
Unused	Loop ID	Opcode

Programming model: Memory map (as viewed from the system bus)

All externally visible state is mapped into a 32-bit (4GB) memory space. Currently the upper 1MB is used for configuration and the remainder gives access to buffer contents.

The following sections must be mapped into memory via the external AXI bus:

- Access to internal buffers
- Control commands (write only)
- Status information (read only)
- Buffer memory map (write only)
- Config register bank (write only)
- Access to internal buffers

Internal Buffers

The architecture supports up to 32 internal buffers currently. The max size of each is 1MB, so the bottom 32MB of the memory map are dedicated to buffer access. Access is read/write, although the correct route to set up a buffer from an external source is with the fill command, since this also sets up the colour and number of target fields.

31:25	24:20	19:2	1:0
0b0000000 (select lower 32MB of memory map)	Buffer ID (0-31)	Buffer offset (1M of 32-bit words)	0b00 (32-bit aligned accesses only)

Control Commands

Writing to this set of registers controls the operation of the accelerator. All registers are write-only. Status information comes from a different part of the memory map (see next section).

31:29	28:25	24:20	19:0
0b111 (select upper 32MB of memory map)	**0b0000 (ID for control commands)**	**Reg ID**	**Value**
		0b00000 LOAD_PC	Start address in internal mem space for program to execute
		0b00001	Any.

		START	Triggers execution at start address
		0b00010 CONTINUE	Any. Triggers restart after halt due to early exit
		0b00011 PAUSE	Any. Stops the fetching of new instructions and the addition of new entries to the scheduler table.
		0b00100 UNPAUSE	Any. Resumes fetching instructions after a PAUSE.

Status Information

Read only information on the state of the accelerator, as accessible to the external (system) bus.

31:30	29:26	25:20	19:0
0b11 (select upper 32MB of memory map)	**0b0001 (ID for status infomation)**	**Reg ID**	**Not used**
		0b000000 CURRENT_PC	Returns current fetch pointer
		0b000001 ACTIVE	Bit 2:0 – acc busy flags, Bit 31 – whole accelerator busy, Bit 30 – accelerator paused on early exit
		0b000001 BUFF_FULL	Vector of flags for buffer fullness
		0b000010 BUFF_BUSY	Vector of flags for buffer busy
		0b000011 BUFF_COLOUR	Vector of flags for buffer colour
		0b100000- 0b111111 Buffer specific status (0-31)	Bit 0 – colour, Bit 1- busy Bit 2 – full Bit 5:3 – num targets remaining

Buffer Memory Map

There are 32 buffers currently supported (this may be reduced through configuration at the hardware compilation level). Each buffer has a size and a start address in the internal memory map of the accelerator (not necessarily the address on the external system bus). These fields re

set up in this section of the memory map. Run-time buffer information (colour, #Targets, filled-status) is only available via the status information registers (above).

31:30	29:26	25	25:20	19:0
0b11 (select upper 32MB of memory map)	**0b0010 (ID for buff memory map)**	**Field select**	**Buff ID (0-31)**	**Value**
		0	0bxxxxx (buff ID)	Start address in internal memory (multiples of 256 bytes, address shifted by 8 internally)
		1	0bxxxxx (buff ID)	Size of buffer in bytes (must be multiple of 32)

Config Register Bank

Each hardware compute block can be set up with a configuration, customising its operation for the current task. This permits a single compute unit to work with a range of activation and weight bit resolutions and select between fully-connected, sparsely-connected or convolutional operation, for example. The interpretation of the config values are unit specific, but the hardware provides a space of configurable size for each such hardware setup.

Writing to a config register is done with two consecutive 32-bit writes. The first one selects the register to be changed and the second is the new value. These values are read by the target accelerator hardware at the point when a task is assigned to that accelerator. The TASK command includes an ID for the required config. No check is made on whether or not the given config is appropriate for the target accelerator. Unpredictable behaviour will result if the target is inappropriate.

31:30	29:26	25	25:20	19:0
0b11 (select upper 32MB of memory map)	**0b0011 (ID for config register map)**	**Field select**	**config ID (0-31)**	**REG ID**
		0	0bxxxxx (config ID)	Bits[19:12] - ID of the config register to be modified (meaning is module specific)
		1	0bxxxxx (config ID)	New value to be written